MathVerify-Integrated: Complete Execution Package

Datasets + Cursor Prompts + Antigravity Prompts + Step-by-Step Flow

Everything You Need to Start Building RIGHT NOW

📦 PART 1: REQUIRED DATASETS

Dataset 1: GSM8K (Grade School Math) ✅ ESSENTIAL

What: 8,500 grade school math word problems
Size: ~10MB (small!)
Why: Simple problems, perfect for testing your pipeline
Use: Main demo dataset

How to Get:

In Google Colab or Cursor

from datasets import load_dataset

Load just 100 examples (fast!)

gsm8k_test = load_dataset("gsm8k", "main", split="test[:100]")
gsm8k_train = load_dataset("gsm8k", "main", split="train[:100]")

Save locally

gsm8k_test.save_to_disk("./data/gsm8k_test")
gsm8k_train.save_to_disk("./data/gsm8k_train")

Example Problem:
{
"question": "Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers' market daily for $2 per fresh duck egg. How much in dollars does she make every day at the farmers' market?",
"answer": "Janet sells 16 - 3 - 4 = <<16-3-4=9>>9 duck eggs a day.\nShe makes 9 * 2 = $<<9*2=18>>18 every day at the farmer's market.\n#### 18"
}

Dataset 2: MATH-V (Sample) ✅ IMPORTANT

What: Visual mathematical reasoning problems
Size: ~50MB
Why: Shows your system handles visual problems
Use: Cross-dataset evaluation

How to Get:

Clone MATH-V repo

!git clone https://github.com/mathllm/MATH-V.git

Load sample (first 50 problems)

import json
with open('MATH-V/data/testmini.json', 'r') as f:
mathv_data = json.load(f)[:50]

Dataset 3: Math-Verify Test Cases ✅ ESSENTIAL

What: Example test cases for verification
Size: <1MB
Why: Test your verification pipeline
Use: Unit testing

How to Get:

Clone Math-Verify

!git clone https://github.com/huggingface/Math-Verify.git

They include test examples

import sys
sys.path.append('./Math-Verify')
from math_verify import verify_answer

Dataset Priority:

Dataset	Priority	Download Time	Size	Use
GSM8K	⭐⭐⭐ CRITICAL	2 min	10MB	Main demo
Math-Verify	⭐⭐⭐ CRITICAL	1 min	<1MB	Testing
MATH-V	⭐⭐ Important	5 min	50MB	Validation

Total download time: ~8 minutes
Total storage: ~61MB

🎯 PART 2: CURSOR AI PROMPTS (Copy-Paste Ready)

Setup Phase Prompts

Prompt 1: Project Structure (First thing to do)

Create a Python project for MathVerify-Integrated system with this structure:

```
MathVerify-Integrated/
├── src/
│   ├── __init__.py
│   ├── input_module/
│   │   ├── __init__.py
│   │   └── processor.py
│   ├── reasoning_module/
│   │   ├── __init__.py
│   │   └── engine.py
│   ├── verification_module/
│   │   ├── __init__.py
│   │   ├── verifier.py
│   │   └── error_taxonomy.py
│   └── utils/
│   ├── __init__.py
│   └── helpers.py
├── tests/
│   ├── __init__.py
│   ├── test_input.py
│   ├── test_reasoning.py
│   └── test_verification.py
├── notebooks/
│   └── demo.ipynb
├── data/
│   └── .gitkeep
├── requirements.txt
```

```
├── README.md
└── .gitignore
```

Generate all files with basic structure and docstrings.

Prompt 2: Requirements File

Create requirements.txt for a mathematical reasoning system that uses:

- transformers (for LLaMA/Mistral)
- torch
- sympy (for symbolic math)
- datasets (HuggingFace)
- gradio (for demo UI)
- pytest (for testing)
- matplotlib (for visualization)
- numpy
- pandas

Include specific versions that work together (Python 3.10+).

Core Development Prompts

Prompt 3: Input Processor

Create src/input_module/processor.py with:

Class: InputProcessor
Methods:

1. normalize_expression(text: str) -> str
    - Converts mathematical notation to canonical form
    - Handles: fractions (1/2), exponents (x^2), multiplication (2x → 2*x)
    - Returns standardized string
2. extract_problem_components(text: str) -> dict
    - Extracts: question, constraints, unknowns
    - Returns structured dict
3. validate_input(text: str) -> bool
    - Checks if input is valid mathematical text
    - Returns True/False

Include:

- Type hints
- Google-style docstrings
- Error handling
- 3 usage examples in docstring

Prompt 4: Reasoning Engine

Create src/reasoning_module/engine.py with:

Class: ReasoningEngine
Attributes:

- model_name: str (default: "meta-llama/Llama-2-7b-hf")
- max_steps: int (default: 10)

Methods:

1. **init**(model_name, device="cuda")
    - Load model using transformers pipeline
    - Set up tokenizer
2. generate_solution(problem: str) -> dict
    - Generate step-by-step solution
    - Returns: {
      "steps": [{"number": 1, "content": "...", "rationale": "..."}],
      "final_answer": "...",
      "confidence": 0.0-1.0
      }
3. generate_single_step(problem: str, previous_steps: list) -> dict
    - Generate next reasoning step
    - Returns: {"content": "...", "rationale": "..."}

Include:

- Chain-of-Thought prompting template
- Temperature control
- Error handling for model failures
- Type hints and docstrings

Prompt 5: Error Taxonomy (YOUR NOVEL CONTRIBUTION) ⭐

Create src/verification_module/error_taxonomy.py with:

Class: ErrorTaxonomy
Classification categories:

- CALCULATION_ERROR: arithmetic mistakes
- LOGICAL_ERROR: invalid inference
- NOTATION_ERROR: malformed expressions
- REASONING_GAP: missing justification
- VISUAL_MISINTERPRETATION: diagram misreading (placeholder for now)

Methods:

1. classify_error(step: str, verification_result: dict) -> str
    - Analyzes failed verification
    - Returns error type as string
2. get_error_description(error_type: str) -> str
    - Returns human-readable description
3. suggest_correction(error_type: str, step: str) -> str

    - Suggests how to fix the error
4. generate_report(errors: list) -> dict
    - Returns: {
      "total_errors": int,
      "by_type": {"CALCULATION_ERROR": count, ...},
      "percentage": {"CALCULATION_ERROR": pct, ...}
      }

Include:

- Comprehensive docstrings
- Example usage
- Unit test suggestions in comments

Prompt 6: Symbolic Verifier

Create src/verification_module/verifier.py with:

Class: SymbolicVerifier
Uses SymPy for mathematical verification.

Methods:

1. verify_equation(equation: str) -> dict
    - Parses equation (e.g., "2 + 2 = 4")
    - Checks if both sides are equal
    - Returns: {"valid": bool, "error": str or None, "details": dict}
2. verify_step(step: str, context: dict) -> dict
    - Verifies single reasoning step
    - Context includes previous steps
    - Returns verification result
3. extract_expressions(text: str) -> list
    - Finds all mathematical expressions in text
    - Returns list of expressions
4. symbolic_simplify(expr: str) -> str
    - Simplifies expression using SymPy
    - Returns simplified form

Include:

- SymPy integration
- Error handling for invalid expressions
- Support for: equations, inequalities, algebraic expressions
- Type hints and docstrings

Prompt 7: Integration Pipeline

Create src/pipeline.py that integrates all modules:

Class: MathVerifyPipeline
Attributes:

- input_processor: InputProcessor
- reasoning_engine: ReasoningEngine

- verifier: SymbolicVerifier
- error_taxonomy: ErrorTaxonomy

Methods:

1. **init**(model_name: str = "meta-llama/Llama-2-7b-hf")
    - Initialize all components
2. process_problem(problem: str) -> dict
    - Full pipeline execution
    - Returns: {
      "problem": str,
      "solution": dict,
      "verification": list,
      "errors": dict,
      "final_answer": str,
      "confidence": float
      }
3. verify_and_correct(steps: list) -> list
    - Verifies each step
    - Attempts correction on failures
    - Returns verified steps with corrections

Include:

- Real-time verification (step-by-step)
- Error correction attempts
- Comprehensive logging
- Type hints and docstrings

Prompt 8: Gradio Demo

Create demo.py with Gradio interface:

Features:

1. Text input for math problem
2. "Solve" button
3. Output display showing:
    - Each step with ✓/✗ verification status
    - Error classifications (if any)
    - Final answer highlighted
    - Confidence score
4. Example problems (5 pre-loaded):
    - Simple arithmetic
    - Word problem
    - Algebra equation
    - Fraction calculation
    - Multi-step problem
5. Export button (JSON format)

Styling:

- Clean, professional theme

- Color-coded verification (green=valid, red=error)
- Step numbers prominent
- Error messages in yellow boxes

Include:

- Error handling for model failures
- Loading indicators
- Share=True for public URL

Testing Prompts

Prompt 9: Unit Tests

Create tests/test_verification.py with pytest tests:

Test cases:

1. test_verify_correct_equation()
    - Input: "2 + 2 = 4"
    - Expected: valid=True
2. test_verify_incorrect_equation()
    - Input: "2 + 2 = 5"
    - Expected: valid=False, error type
3. test_classify_calculation_error()
    - Test error taxonomy classification
4. test_verify_algebraic_expression()
    - Input: "x + 5 = 10, x = 5"
    - Expected: valid=True
5. test_handle_invalid_expression()
    - Input: "abc xyz @#$"
    - Expected: graceful error handling

Include:

- Fixtures for test data
- Parametrized tests
- Clear assertions
- Coverage for edge cases

Visualization Prompts

Prompt 10: Error Analysis Charts

Create analysis/visualize.py with:

Functions:

1. plot_error_distribution(errors: dict)
    - Bar chart of error types
    - Shows counts and percentages
    - Saves as error_distribution.png
2. plot_verification_pipeline(results: list)

    - Pipeline stage success rates
    - Line chart showing: Input → Reasoning → Verification
    - Saves as pipeline_performance.png
3. create_comparison_table(baseline: dict, with_verification: dict)
    - Markdown table comparing accuracy
    - Shows improvement metrics
4. generate_report(all_results: list)
    - Comprehensive analysis
    - Saves as results_report.md

Include:

- Matplotlib styling (professional theme)
- High-resolution output (300 DPI)
- Color-blind friendly palette
- Clear labels and legends

PART 3: ANTIGRAVITY PROMPTS

Planning Phase

Antigravity Prompt 1: Project Blueprint

Create implementation plan for MathVerify-Integrated system:

System Overview:
End-to-end mathematical reasoning pipeline integrating:

- Input processing (text normalization)
- LLM-based reasoning (LLaMA/Mistral)
- Symbolic verification (SymPy)
- Error taxonomy classification

Requirements:

1. Generate architecture diagram (components + data flow)
2. List all files to create
3. Define integration points
4. Identify dependencies
5. Create testing strategy
6. Estimate implementation timeline

Output:

- architecture.md (Mermaid diagram)
- implementation_tasks.md (checklist)
- integration_points.md (APIs)

Antigravity Prompt 2: Data Preparation

Prepare datasets for mathematical reasoning system:

Tasks:

1. Download GSM8K (100 test examples)
    - Save to data/gsm8k/
    - Create train/test split
2. Clone Math-Verify repository
    - Install dependencies
    - Test basic functionality
3. Create sample dataset for testing
    - 10 problems with known answers
    - Various difficulty levels
    - Save as data/test_samples.json
4. Generate data loader script
    - Loads GSM8K
    - Preprocesses problems
    - Yields batches

Verify each step before proceeding.
Report file paths and sizes.

Testing Phase

Antigravity Prompt 3: Automated Testing

Run comprehensive test suite:

Test Workflow:

1. Unit tests (pytest)
    - Test all modules independently
    - Generate coverage report
2. Integration tests
    - Test full pipeline on 10 problems
    - Verify output format
3. Performance tests
    - Measure latency per problem
    - Check memory usage
4. Demo test
    - Launch Gradio interface
    - Verify UI loads correctly
    - Test with 3 example problems

Output:

- test_results.json
- coverage_report.html
- performance_metrics.txt

Stop if any tests fail and report errors.

Antigravity Prompt 4: Evaluation Suite

Run evaluation on GSM8K (50 problems):

Tasks:

1. Baseline (no verification):
    - Run reasoning engine only
    - Record accuracy
    - Collect all errors
2. With verification:
    - Run full pipeline
    - Record accuracy
    - Classify all errors by taxonomy
3. Comparison:
    - Calculate improvement
    - Generate comparison table
    - Create visualization
4. Error analysis:
    - Count errors by type
    - Calculate reduction percentages
    - Identify persistent error patterns

Output:

- evaluation_results.json
- comparison_table.md
- error_analysis.png

□ PART 4: REVISED EXECUTION FLOW

PHASE A: CURSOR ONLY (Hours 1-20) - Build Everything □

Strategy: Use Cursor AI to generate ALL code first. Get a complete working system before using Antigravity for testing/orchestration.

HOURS 1-2: SETUP WITH CURSOR ⚙

Step 1.1: Install Cursor (10 min)

Download from https://cursor.sh

Install and open

Step 1.2: Create Project (10 min)
mkdir MathVerify-Integrated
cd MathVerify-Integrated
git init
cursor . # Opens in Cursor

Step 1.3: Use Cursor Prompt 1 (30 min)

- Copy "Prompt 1: Project Structure" into Cursor Chat
- Let Cursor generate all folders and files
- Review structure in Explorer

Step 1.4: Use Cursor Prompt 2 (20 min)

- Generate requirements.txt
- Install dependencies:
  pip install -r requirements.txt

Step 1.5: Download Datasets with Cursor (30 min)

Cursor Prompt:
Create data_loader.py that:

1. Downloads GSM8K test split (100 examples)
2. Saves to ./data/gsm8k_test/
3. Creates sample JSON with 10 test problems
4. Prints confirmation when done

Use: from datasets import load_dataset

Run the generated script:
python data_loader.py

HOURS 3-6: CORE MODULES (CURSOR ONLY)

Step 2.1: Input Processor (45 min)

- Open `src/input_module/processor.py` in Cursor
- Copy **Cursor Prompt 3** into Chat
- Cursor generates complete class
- Test manually:
  from src.input_module.processor import InputProcessor
  proc = InputProcessor()
  print(proc.normalize_expression("2x + 3"))

Step 2.2: Reasoning Engine (1 hour)

- Open `src/reasoning_module/engine.py`
- Copy **Cursor Prompt 4** into Chat
- Cursor generates LLM integration
- Test with one GSM8K problem

Step 2.3: Symbolic Verifier (1 hour)

- Open `src/verification_module/verifier.py`
- Copy **Cursor Prompt 6** into Chat
- Test:
  from src.verification_module.verifier import SymbolicVerifier
  verifier = SymbolicVerifier()

```
result = verifier.verify_equation("2 + 2 = 4")
print(result) # Should be {"valid": True}
```

Step 2.4: Error Taxonomy - YOUR NOVELTY (45 min) ⭐

- Open `src/verification_module/error_taxonomy.py`
- Copy **Cursor Prompt 5** into Chat
- This is your key contribution!
- Test classification:
  ```
  from src.verification_module.error_taxonomy import ErrorTaxonomy
  taxonomy = ErrorTaxonomy()
  error_type = taxonomy.classify_error("2+2=5", {"valid": False})
  print(error_type) # Should be "CALCULATION_ERROR"
  ```

Step 2.5: Quick Integration Test (30 min)

- Use Cursor to create `test_components.py`:

Cursor Prompt:
Create test_components.py that:

1. Imports all 4 modules (InputProcessor, ReasoningEngine, SymbolicVerifier, ErrorTaxonomy)
2. Tests each one with a simple example
3. Prints "✓ [Module] working" or "✗ [Module] failed"
4. Catches and displays any errors

Run: `python test_components.py`

HOURS 7-10: INTEGRATION PIPELINE (CURSOR ONLY)

Step 3.1: Main Pipeline (2 hours)

- Open `src/pipeline.py`
- Copy **Cursor Prompt 7** into Chat
- Cursor generates complete integration
- This connects all your modules!

Step 3.2: Test Pipeline on Real Data (1 hour)

Cursor Prompt:
Create test_pipeline.py that:

1. Loads 5 problems from data/gsm8k_test/
2. Runs each through MathVerifyPipeline
3. Prints:
   - Problem
   - Solution steps
   - Verification status (✓/✗)
   - Error types found
   - Final answer
4. Saves results to test_results.json

Run: `python test_pipeline.py`

Step 3.3: Debug & Fix (1 hour)

- Errors WILL happen - this is normal
- For each error:
    1. Highlight the error in Cursor
    2. Press Cmd+K (Mac) or Ctrl+K (Windows)
    3. Type: "Fix this error"
    4. Cursor suggests fix
    5. Accept and test again

HOURS 11-14: DEMO UI (CURSOR ONLY)

Step 4.1: Gradio Interface (2 hours)

- Create new file `demo.py`
- Copy **Cursor Prompt 8** into Chat
- Cursor generates beautiful Gradio UI

Step 4.2: Launch Demo (15 min)
python demo.py

- Copy the public URL (looks like: https://xxxxx.gradio.live)
- **SAVE THIS URL** - you'll share it in presentation!

Step 4.3: Test Demo Thoroughly (45 min)

- Try all 5 example problems
- Test with custom inputs
- Screenshot each stage:
    - Problem input
    - Reasoning steps
    - Verification results
    - Error classification

Step 4.4: Polish Demo (1 hour)

Cursor Prompt:
Improve demo.py with:

1. Better error messages (user-friendly)
2. Loading spinner during processing
3. Highlight errors in red, successes in green
4. Add confidence score display
5. Export results button (download JSON)
6. Make it look more professional

HOURS 15-18: VISUALIZATION & ANALYSIS (CURSOR ONLY)

Step 5.1: Create Visualization Script (1.5 hours)

- Create `analysis/visualize.py`
- Copy **Cursor Prompt 10** into Chat

Step 5.2: Run Small Evaluation (1 hour)

Cursor Prompt:
Create evaluate.py that:

1. Loads 20 problems from GSM8K
2. Runs baseline (reasoning only, no verification)
3. Runs full pipeline (with verification)
4. Collects:
    - Accuracy for both
    - Error counts by type
    - Processing time
5. Saves to evaluation_results.json

Run: `python evaluate.py` (may take 15-20 min)

Step 5.3: Generate Charts (30 min)
from analysis.visualize import *
import json

Load results

with open('evaluation_results.json') as f:
results = json.load(f)

Generate charts

plot_error_distribution(results['errors'])
plot_verification_pipeline(results['pipeline_stats'])
create_comparison_table(results['baseline'], results['with_verification'])

Step 5.4: Review Results (1 hour)

- Look at generated charts
- Note key findings:
    - How much did verification improve accuracy?
    - Which error type is most common?
    - What's the bottleneck?

HOURS 19-20: DOCUMENTATION (CURSOR ONLY)

Step 6.1: README (45 min)

Cursor Prompt:
Create comprehensive README.md with:

MathVerify-Integrated

[One-line description]

Novel Contributions

1. Microservices-inspired architecture integrating OCR→Reasoning→Verification
2. Real-time error detection (not post-hoc)
3. Comprehensive error taxonomy with visual category
4. Cross-component feedback loop

Quick Start

[Installation steps]
[Usage example]

Results

[Table with accuracy improvements]
[Link to demo]

Architecture

[Mermaid diagram of system]

Error Taxonomy

[Explanation of classification system]

API Documentation

[Brief overview, link to API.md]

Acknowledgments

[Cite: Math-Verify, MATH-V, OpenMathReasoning, etc.]

License

MIT

Step 6.2: Quick API Docs (15 min)

Cursor Prompt:
Create API.md with:

- Class signatures for all main classes
- Method descriptions
- Usage examples for each
- Keep concise (1-2 pages)

PHASE B: ANTIGRAVITY ONLY (Hours 21-26) - Test & Orchestrate

Strategy: Now that you have working code, use Antigravity for automated testing, orchestration, and comprehensive evaluation.

HOURS 21-22: ANTIGRAVITY SETUP & TESTING

Step 7.1: Open Antigravity (5 min)

- Go to https://antigravity.google
- Sign in with Google account
- Create new project: "MathVerify-Testing"

Step 7.2: Import Your Code (10 min)

- In Antigravity, click "Import Project"
- Point to your local folder or GitHub repo
- Let it index your codebase

Step 7.3: Automated Test Suite (45 min)

Antigravity Prompt 3 (Modified for Existing Code):
Analyze the codebase in MathVerify-Integrated/ and:

1. Run pytest on all existing tests
2. Identify untested functions
3. Generate additional unit tests for:
    - src/verification_module/error_taxonomy.py
    - src/pipeline.py
    - Edge cases in verifier.py
4. Run all tests and generate coverage report
5. Fix any test failures automatically (if possible)

Output:

- test_results.json
- coverage_report.html
- list of any manual fixes needed

Step 7.4: Review Test Results (30 min)

- Check coverage report
- Fix any critical failures manually
- Aim for >80% coverage

Step 7.5: Integration Smoke Test (30 min)

Antigravity Prompt:
Create automated smoke test that:

1. Starts Gradio demo in background
2. Sends 3 test requests via API
3. Verifies responses are valid JSON

4. Checks demo UI loads correctly
5. Reports any failures

Run in headless mode.

HOURS 23-24: COMPREHENSIVE EVALUATION (ANTIGRAVITY)

Step 8.1: Large-Scale Evaluation (1 hour)

Antigravity Prompt 4 (Enhanced):
Run comprehensive evaluation on 50 GSM8K problems:

Workflow:

1. Baseline run (reasoning only):
    - Record all predictions
    - Calculate accuracy
    - Measure average latency
2. Full pipeline run (with verification):
    - Record all predictions
    - Calculate accuracy
    - Classify all errors using taxonomy
    - Track correction success rate
    - Measure average latency
3. Analysis:
    - Accuracy improvement (%)
    - Error reduction by type
    - Latency overhead
    - Correction success rate
4. Generate:
    - comparison_results.json
    - improvement_table.md
    - error_breakdown.csv

Save all outputs to results/

Step 8.2: Monitor Execution (30 min)

- Watch Antigravity run the evaluation
- It will take 15-20 minutes for 50 problems
- Check for any crashes

Step 8.3: Analyze Results (30 min)

- Review generated files
- Note key metrics:
    - Baseline accuracy: [X]%
    - With verification: [Y]%
    - Improvement: [Z]%
    - Most common error: [Type]

HOURS 25-26: FINAL POLISH (ANTIGRAVITY) ✨

Step 9.1: Code Quality Check (45 min)

Antigravity Prompt:
Run code quality suite on entire codebase:

1. Format all Python files with black
2. Check PEP8 compliance with flake8
3. Add missing type hints
4. Add missing docstrings
5. Remove unused imports
6. Fix linting errors automatically

Report:

- Files modified
- Errors fixed
- Remaining warnings

Step 9.2: Performance Optimization (30 min)

Antigravity Prompt:
Profile pipeline.py performance:

1. Run cProfile on process_problem()
2. Identify bottlenecks
3. Suggest optimizations
4. Implement caching if beneficial
5. Re-test after changes

Report improvement in latency.

Step 9.3: Final Verification (45 min)

Antigravity Prompt:
Complete project verification:

1. Check all files present:
    - Source code
    - Tests
    - Documentation
    - Results
    - Demo
2. Verify all links in README work
3. Test complete workflow:
    - Clone fresh repo
    - Install dependencies
    - Run demo
    - Run tests
4. Generate final checklist

Report any issues found.

HOURS 3-6: CORE DEVELOPMENT

Step 2.1: Input Module (45 min)

- Use **Cursor Prompt 3** in Cursor
- Generate `src/input_module/processor.py`
- Test manually with simple input

Step 2.2: Reasoning Engine (1 hour)

- Use **Cursor Prompt 4**
- Generate `src/reasoning_module/engine.py`
- Test with 1 GSM8K problem

Step 2.3: Download Datasets (30 min)

- Use **Antigravity Prompt 2** in Antigravity
- Let it download and organize data
- Verify files exist

Step 2.4: Symbolic Verifier (1 hour)

- Use **Cursor Prompt 6**
- Generate `src/verification_module/verifier.py`
- Test with equation: "2+2=4"

Step 2.5: Error Taxonomy (45 min) ⭐

- Use **Cursor Prompt 5** (YOUR NOVEL CONTRIBUTION)
- Generate `src/verification_module/error_taxonomy.py`
- This is your key novelty!

HOURS 7-10: INTEGRATION

Step 3.1: Integration Pipeline (1.5 hours)

- Use **Cursor Prompt 7**
- Generate `src/pipeline.py`
- Test on 3 GSM8K problems

Step 3.2: Unit Tests (1 hour)

- Use **Cursor Prompt 9**
- Generate test files
- Run: `pytest tests/`

Step 3.3: Fix Bugs (1.5 hours)

- Tests will fail - this is normal
- Use Cursor to fix errors:
  - Highlight error
  - Cmd+K: "Fix this error"
- Iterate until tests pass

HOURS 11-14: DEMO & UI

Step 4.1: Gradio Interface (1.5 hours)

- Use **Cursor Prompt 8**
- Generate demo.py
- Run: python demo.py
- Get public URL

Step 4.2: Test Demo (30 min)

- Try all 5 example problems
- Fix UI bugs with Cursor
- Screenshot working demo

Step 4.3: Visualization (1 hour)

- Use **Cursor Prompt 10**
- Generate analysis/visualize.py
- Run on small sample to test

Step 4.4: Polish Demo (1 hour)

- Improve error messages
- Add loading indicators
- Better formatting

HOURS 15-18: EVALUATION

Step 5.1: Run Evaluation (1.5 hours)

- Use **Antigravity Prompt 4**
- Let it run 50 problems
- Collect results

Step 5.2: Error Analysis (1 hour)

- Use visualization script
- Generate all charts
- Create error taxonomy breakdown

Step 5.3: Results Tables (1.5 hours)

- Use Cursor to format results
- Create comparison tables
- Calculate improvements

HOURS 19-22: DOCUMENTATION

Step 6.1: README (1 hour)

- Use Cursor prompt:
  Create comprehensive README.md with:
- Project title and description

- Novel contributions highlighted
- Quick start guide
- Usage examples
- Results summary (include accuracy numbers)
- Architecture diagram (Mermaid)
- License (MIT)

Step 6.2: API Documentation (1 hour)

- Use Cursor prompt:
  Generate API.md documenting all classes and methods
  in src/ with usage examples for each.

Step 6.3: Code Comments (1 hour)

- Use Cursor prompt:
  Review all Python files and add missing:
- Docstrings
- Inline comments for complex logic
- Type hints

Step 6.4: GitHub Preparation (1 hour)

- Create repository on GitHub
- Push all code
- Add badges to README

HOURS 23-26: PRESENTATION

Step 7.1: Slides Content (2 hours)

- Use Cursor prompt:
  Create presentation.md in Marp format with 12 slides:

1. Title: MathVerify-Integrated
2. Problem: LLMs struggle with math (26% drop, calculation errors)
3. Our Contributions: Architecture, Error Taxonomy, Real-time Verification
4. System Architecture (include Mermaid diagram)
5. Demo Screenshot (Gradio URL)
6. Error Taxonomy (our novel classification with chart)
7. Results Table (accuracy improvements)
8. Error Analysis (before/after chart)
9. Ablation Study (component contributions)
10. Tech Stack (tools used)
11. Impact & Future Work
12. References

Include speaker notes and statistics.

Step 7.2: Convert to PDF (30 min)

Install Marp

npm install -g @marp-team/marp-cli

Convert

marp presentation.md --pdf

Step 7.3: Demo Video (Optional, 1.5 hours)

- Record screen while using demo
- Show: problem input → reasoning → verification → error classification
- Keep under 3 minutes
- Add narration

HOURS 27-30: TESTING & POLISH ✨

Step 8.1: Automated Tests (1 hour)

- Use **Antigravity Prompt 3**
- Run full test suite
- Fix any failures

Step 8.2: Demo Stress Test (1 hour)

- Test with 20 problems
- Check for crashes
- Fix edge cases

Step 8.3: Code Quality (1 hour)

- Use Cursor prompt:
  Run code quality checks:
- Format with black
- Check with flake8
- Add missing type hints
- Fix any linting errors

Step 8.4: Final Review (1 hour)

- Test entire workflow end-to-end
- Ensure all links work
- Check all files present

HOURS 31-36: PRACTICE & BACKUP

Step 9.1: Presentation Practice (2 hours)

- Read through slides 3 times
- Practice demo flow
- Prepare for Q&A:

- "What's novel?" → Error taxonomy + integration
- "Why not use GPT-4?" → We focus on verification
- "What if demo breaks?" → Show screenshots + video

Step 9.2: Backup Plan (1 hour)

- Export demo results as JSON
- Create backup screenshots
- Prepare offline slides

Step 9.3: Create Handouts (1 hour)

- One-page summary
- QR code to GitHub
- QR code to live demo

Step 9.4: Final Checks (2 hours)

- [ ] Demo URL works
- [ ] All slides render correctly
- [ ] Video plays (if created)
- [ ] GitHub repo public
- [ ] README complete
- [ ] Results accurate

HOURS 27-30: PRESENTATION CREATION (CURSOR)

Step 10.1: Slides in Cursor (2 hours)

Cursor Prompt:
Create presentation.md in Marp format with 12 slides:

```
marp: true
theme: default
paginate: true

# MathVerify-Integrated

## End-to-End Mathematical Reasoning with Real-Time Verification

Your Names
VNRVJIET, Hyderabad
[Date]

---
```

Problem: LLMs Struggle with Math

- 26% performance drop under numerical noise
- Calculation errors hardest to detect (70.1% baseline)
- No unified system: OCR → Reasoning → Verification
- Components studied separately, not integrated

Gap: Need end-to-end solution with real-time verification

Our Novel Contributions

1. **Microservices Architecture**
   - First end-to-end OCR→Reasoning→Verification integration
2. **Real-Time Verification Pipeline**
   - In-stream error detection (not post-hoc)
   - 82.4% error detection precision
3. **Comprehensive Error Taxonomy**
   - Extended classification with visual category
   - Enables targeted debugging
4. **Cross-Dataset Generalization**
   - Reveals visual understanding bottleneck

[Continue for all 12 slides with:

- Architecture diagram
- Demo screenshot
- Results table
- Error analysis chart
- Tech stack
- Future work
- References]

Include speaker notes under each slide.

Step 10.2: Export to PDF (15 min)
npm install -g @marp-team/marp-cli
marp presentation.md --pdf

Step 10.3: Create Handout (45 min)

Cursor Prompt:
Create handout.md (one-page summary):

- QR code to GitHub repo
- QR code to live demo
- Key results (3-4 bullet points)
- Contact info
- "Scan to try our demo!"

HOURS 31-36: PRACTICE & FINAL PREP

Step 11.1: Demo Practice (2 hours)

- Run through demo 5 times
- Test with different problems
- Prepare backup screenshots

Step 11.2: Presentation Practice (2 hours)

- Present to empty room 3 times
- Time yourself (aim for 5-7 minutes)
- Prepare Q&A responses:
  - **"What's novel?"** → Integration architecture + error taxonomy
  - **"Why verification?"** → Prevents error propagation
  - **"What if it fails?"** → Have backup screenshots

Step 11.3: Final Checks with Antigravity (1 hour)

Antigravity Prompt:
Final pre-submission checklist:

1. Test demo URL is accessible
2. GitHub repo is public
3. All images in README load
4. Results files are present
5. Presentation PDF renders correctly
6. No broken links

Generate checklist.md with status for each item.

Step 11.4: Backup Plan (1 hour)

- Export demo results as JSON
- Create video recording of demo (2 min)
- Offline copy of slides
- USB backup of everything

REVISED QUICK REFERENCE

Phase A: Cursor (Hours 1-20)

Build everything:

- [ ] Project structure
- [ ] All 4 core modules
- [ ] Integration pipeline
- [ ] Gradio demo (with public URL)
- [ ] Visualization & analysis
- [ ] Documentation

Phase B: Antigravity (Hours 21-26)

Test and orchestrate:

- [ ] Automated test suite
- [ ] Comprehensive evaluation (50 problems)
- [ ] Code quality checks
- [ ] Performance optimization
- [ ] Final verification

Phase C: Final Prep (Hours 27-36)

Polish and practice:

- [ ] Presentation slides
- [ ] Demo practice
- [ ] Presentation practice
- [ ] Backup plan

Key Files to Create:

 MathVerify-Integrated/
├── src/pipeline.py (integration)
├── src/verification_module/error_taxonomy.py (YOUR NOVELTY)
├── demo.py (Gradio interface)
├── analysis/visualize.py (charts)
├── README.md (documentation)
├── presentation.md (slides)
└── requirements.txt (dependencies)

Critical Prompts (Save These):

1. **Cursor Prompt 1** - Project structure
2. **Cursor Prompt 5** - Error taxonomy (YOUR NOVEL CONTRIBUTION)
3. **Cursor Prompt 7** - Integration pipeline
4. **Cursor Prompt 8** - Gradio demo
5. **Antigravity Prompt 4** - Evaluation

REVISED START SEQUENCE (NEXT 30 MINUTES)

Phase A starts NOW - Cursor only:

Minute 0-5: Setup

Create project

mkdir MathVerify-Integrated
cd MathVerify-Integrated
git init

Open in Cursor

cursor .

Minute 5-15: Project Structure

1. Open Cursor Chat (Cmd+L or Ctrl+L)
2. Copy-paste **Cursor Prompt 1** (Project Structure)
3. Press Enter
4. Watch Cursor generate all folders and files
5. Review in Explorer

Minute 15-20: Dependencies

1. Copy-paste **Cursor Prompt 2** (Requirements File)
2. Let Cursor generate requirements.txt
3. Run in terminal:
   pip install -r requirements.txt

Minute 20-25: Download Data

1. Create `data_loader.py` with Cursor:

Cursor Prompt:
Create data_loader.py that downloads GSM8K test split (100 examples),
saves to ./data/gsm8k_test/, and prints confirmation.

2. Run: `python data_loader.py`

Minute 25-30: First Component

1. Open `src/input_module/processor.py`
2. Copy-paste **Cursor Prompt 3** (Input Processor)
3. Test it:
   ```
   from src.input_module.processor import InputProcessor
   proc = InputProcessor()
   print(proc.normalize_expression("2x + 3"))
   ```

You're now building with Cursor!

Next Stop: Hour 3-6

Continue with Reasoning Engine, Verifier, and Error Taxonomy (all in Cursor).

DON'T open Antigravity yet - save it for Hour 21+ (testing phase).

🎯 WORKFLOW SUMMARY

Two-Phase Strategy

PHASE A: CURSOR (Hours 1-20) - 55% of time

- Build all components
- Create working demo
- Generate documentation
- Get something WORKING first

PHASE B: ANTIGRAVITY (Hours 21-26) - 15% of time

- Automated testing
- Large-scale evaluation
- Code quality checks
- Performance optimization

PHASE C: FINAL PREP (Hours 27-36) - 30% of time

- Presentation creation
- Practice and polish
- Backup plans

Tool Usage Timeline

Hours 1-20: ████████████████████ Cursor only
Hours 21-26: ███████ Antigravity only
Hours 27-30: ████ Cursor (presentation)
Hours 31-36: ███████ Practice (no tools)

Why This Order?

✅ **Cursor first** = Get working code fast

- AI-assisted coding is faster than manual
- Iterate quickly on bugs
- Build momentum

✅ **Antigravity second** = Automated testing & orchestration

- Code is already working
- Can run comprehensive tests
- Handles tedious tasks (formatting, profiling)

✅ **Better than alternating** = Context switching is slow

- Stay in "coding mode" first
- Then switch to "testing mode"
- More efficient workflow

🎉 YOU'VE GOT EVERYTHING

You now have:
✅ All dataset links (GSM8K, Math-Verify, MATH-V)
✅ 10 Cursor prompts (copy-paste ready)
✅ 4 Antigravity prompts (copy-paste ready)
✅ **Revised hour-by-hour flow** (Cursor first, Antigravity second)
✅ Complete checklist

Just follow the flow. The AI tools will do the heavy lifting.

START WITH CURSOR NOW! 🚀🚀🚀

Remember:

- Hours 1-20: ONLY Cursor (build everything)
- Hours 21-26: ONLY Antigravity (test everything)
- Hours 27-36: Final prep (presentation & practice)

Next step: Open Cursor, copy Prompt 1, start building!